

Deep Reinforcement Learning for Combinatorial Optimization

A tutorial at IJCAI-ECAI '26

About organizers



Dr. Zaharah Bukhsh
Asst. professor



Dr. Yaoxin Wu
Asst. professor



Dr. Yingqian Zhang
Asso. professor



Jesse van
Remmerden
PhD candidate



Yue Yu
PhD candidate

The tutorial is based on
book chapter

Deep Reinforcement Learning for Combinatorial Optimization



<https://research.tue.nl/en/publications/deep-reinforcement-learning-for-combinatorial-optimization/>

Agenda

Part 1 - Theoretical foundation

- Intro (5min) -ZB
- Why DRL for combinatorial optimization? (5 min) -YZ
- Design pillars to develop end-to-end DRL solution (35 min) -ZB
- DRL for Classical COPs (VRP and JSP) (20min) - YW
- Beyond Classical VRP and JSP (15min) - YW
- Q&A (5 min) - YW+ZB?
- Slack (5min)

Part 2 - Hands-on exercises

- DRL for real-world problems (20min) -YZ
- Open challenges and takeaways (10min) -ZB
- Code walkthrough - Job Shop Scheduling Problem (JSP) (30 min) - JvR
- Code walkthrough - Capacitated Vehicle Routing Problem (CVRP) - (30 min) - YY

Agenda

Part 1 - Theoretical foundation

- Why DRL for combinatorial optimization?
- Design pillars to develop end-to-end DRL solution
- DRL for Classical COPs (VRP and JSP)
- Beyond Classical VRP and JSP
- Q&A

Background and motivation

Why DRL for combinatorial optimization?

Combinatorial optimization problems & their variants in practice

Combinatorial optimization problems:

Often NP-hard, with objectives, constraints

Route planning for package delivery

- Vehicle routing problem (VRP)

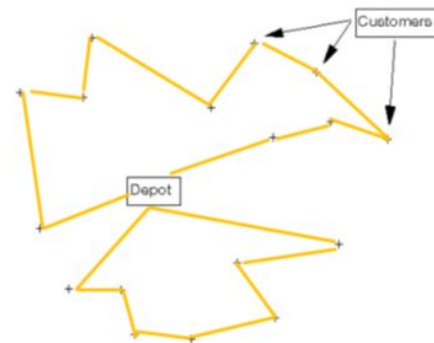
Stacking/packing problems in harbours/warehouses;

Train shunting

- (3D) bin-packing problem

Scheduling jobs on production lines

- Job shop scheduling



Example: Vehicle Routing problem with Time windows (VRPTW) problem

Optimization solver

Domain/expert knowledge

subject to

$$\begin{aligned}
 \text{[VRPTW]} \quad & \min \sum_{k \in K} \sum_{(i,j) \in A} c_{ij} x_{ijk} \\
 \text{subject to} \quad & \sum_{k \in K} \sum_{(i,j) \in A} x_{ijk} = 1, \quad \forall i \in N \\
 & \sum_{j \in N: (i,j) \in A} x_{ijk} = 1, \quad \forall k \in K \\
 & \sum_{i \in \Delta^+(j)} x_{ijk} - \sum_{i \in \Delta^+(j)} x_{ikj} = 0, \quad \forall k \in K, \forall j \in N \\
 & \sum_{i \in \Delta^-(i_0+1)} x_{ik,i_0+1} = 1, \quad \forall k \in K \\
 & w_{ikj}/w_0 + s_i + t_{ij} - w_{jk} \leq 0, \quad \forall k \in K, \forall (i,j) \in A \\
 & w_0 \left(\sum_{j \in \Delta^+(i_0)} x_{ijk} \right) \leq w_{ik} \leq b_i \left(\sum_{j \in \Delta^-(i_0)} x_{ikj} \right), \quad \forall k \in K, \forall i \in N \\
 & R \leq w_{ik} \leq L, \quad \forall k \in K, \forall i \in \{0, n+1\} \\
 & \sum_{i \in N} d_i \sum_{j \in \Delta^-(i)} x_{ijk} \leq C, \quad \forall k \in K \\
 & x_{ijk} \geq 0, \quad \forall k \in K, \forall (i,j) \in A \\
 & x_{ijk} \text{ binary}, \quad \forall k \in K, \forall (i,j) \in A.
 \end{aligned}$$

Best route: hub-> A
-> C -> ... -> hub

What could possibly go wrong?



Picture from <https://ecommercegermany.com>

Challenges of "traditional" optimization algorithms

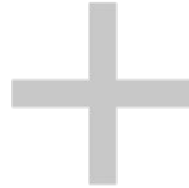
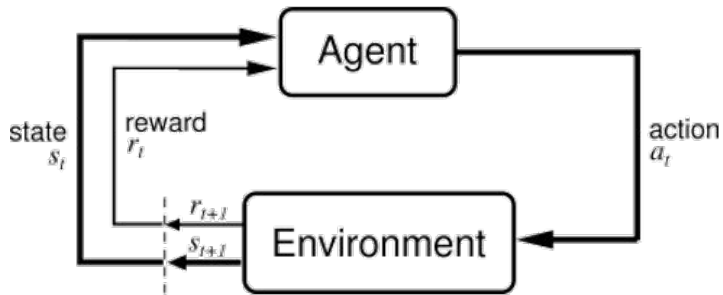
- Hard to handle uncertainties and dynamics
- Hard to model different situations (weekend, weekday...)
- unknown relations (weather vs travel time...)

Traditional Optimization vs Machine Learning

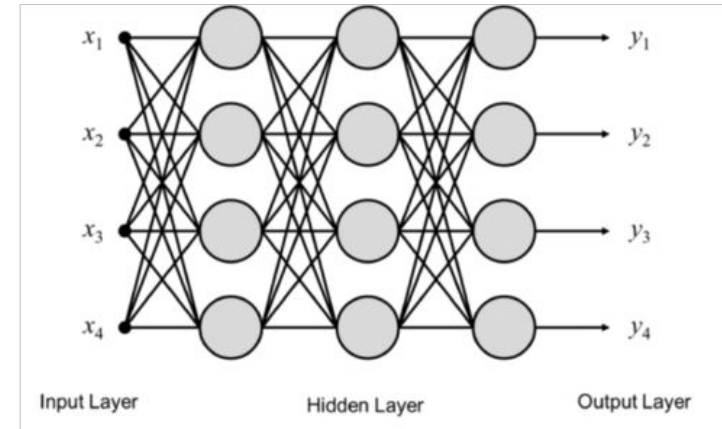
	Traditional algorithms	Machine learning based
Uncertainty	Needs explicitly uncertainty models	Can learn uncertainty patterns from data and experience
Dynamic environments	Requires specialized online algorithms or re-optimization	Can learn decisions that respond quickly to changing environment
Multimodal and high-dimensional inputs	Hard to incorporate unstructured data	Can learn representations from multimodal inputs, e.g., images, text, graphs, sensor data...
Scalability and real-time	Can be computationally expensive	Fast inference enables large-scale and real-time decision making
Adaptation	Typically designed for general problems	Can learn to solve particular instances better
Cross-instance learning	Hard to learn from previously solved instances	Can learn patterns and transfer knowledge across previous instances

Deep Reinforcement Learning

Reinforcement Learning



Deep Learning



Deep Reinforcement Learning

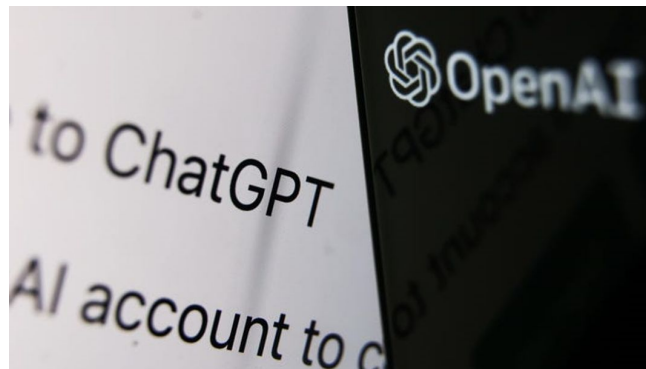
Success of deep reinforcement learning



Atari games: human-level control through DRL (2015)



AlphaGo beats Go master Lee Se-dol (2016)



Hard combinatorial optimization problems?



source: NS



source: Wefabricate



source: vanderlande

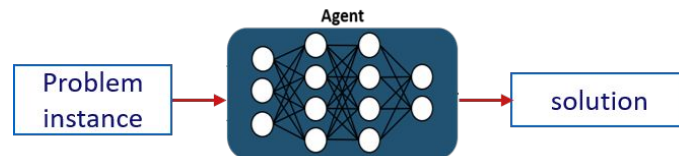
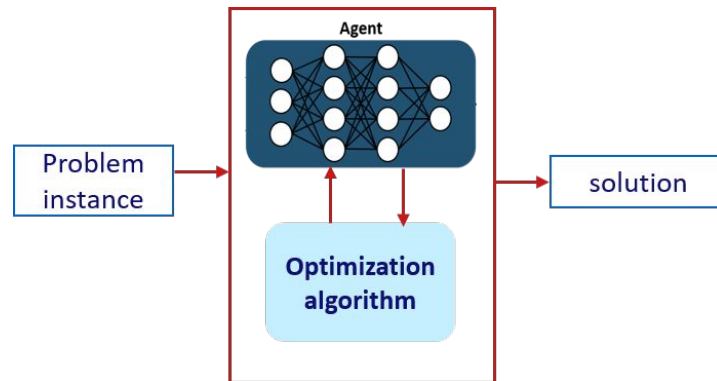
DRL for COPs

Hybrid learning approaches

- DRL-assisted IP: learning cut plane selection, variable selection ...
- DRL-assisted heuristics: Learning to repair/configure operators in local search and evolutionary algorithms ...

End-to-end (E2E) approaches

- DRL agent learns to construct solutions directly (Vinyals et al., 2015, Bello et al., 2016, Kool et al., 2019)



How to develop end-to-end DRL solution for COP

Four design pillars

From COP to sequential decision-making

A COP instance is a tuple $p = (X_p, f_p, C_p)$,

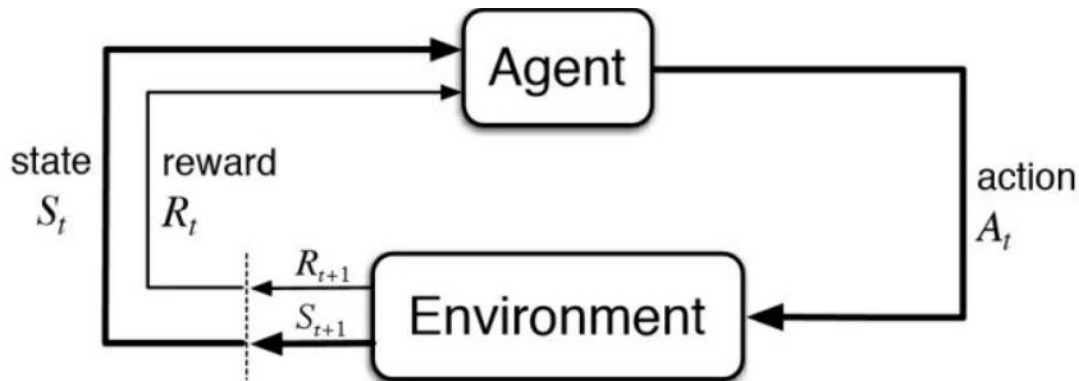
where

- X_p denotes the solution space, i.e., the set of all possible solutions,
- $f_p : X_p \rightarrow \mathbb{R}$ is the objective function to be minimized (or maximized),
- $C_p = \{c_1, c_2, \dots, c_{m_p}\}$ is the set of constraints, with each $c_i : X_p \rightarrow \{0, 1\}$ indicating whether a solution satisfies the i -th constraint.

- For a given instance p , an optimal solution x_p^* must satisfy two conditions:
 - feasibility: $x_p^* \in X_{C_p}$
 - optimality (for minimization) $f_p(x_p^*) \leq f_p(x)$

From COP to sequential decision-making

- Reinforcement learning is a sequential decision-making paradigm.
- It is about learning what to do – how to map observations to actions– to maximize the reward.



RL agent-environment loop.

From COP to sequential decision-making

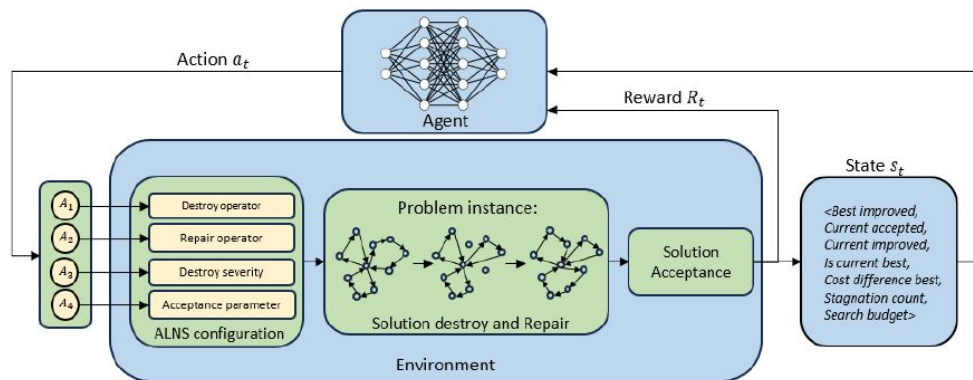
- COPs maps to sequential decision-making paradigm where an agent can either **learn-to-improve** solutions or **learn-to-construct** solutions.

From COP to sequential decision-making

- COPs maps to sequential decision-making paradigm where an agent can either **learn-to-improve** solutions or **learn-to-construct** solutions.

Learn-to-Improve

- Starts from an existing complete solution
- Learns to apply local moves to refine it
- Reward = improvement over the best-so-far (incumbent) solution
- Runs for a fixed step budget T , keeps the best solution seen



Reijnen, R., Zhang, Y., Lau, H. C., & Bukhsh, Z. (2024, May). Online control of adaptive large neighborhood search using deep reinforcement learning. In *Proceedings of the international conference on automated planning and scheduling* (Vol. 34, pp. 475-483).

From COP to sequential decision-making

- COPs maps to sequential decision-making paradigm in broadly two ways:

Learn-to-Construct

- Starts from an empty / trivial state
- Iteratively adds components until a complete solution is formed
- State = static instance info + current partial solution
- Episode ends once the solution is complete
- The most natural mapping from CO to MDP

Learn-to-Improve

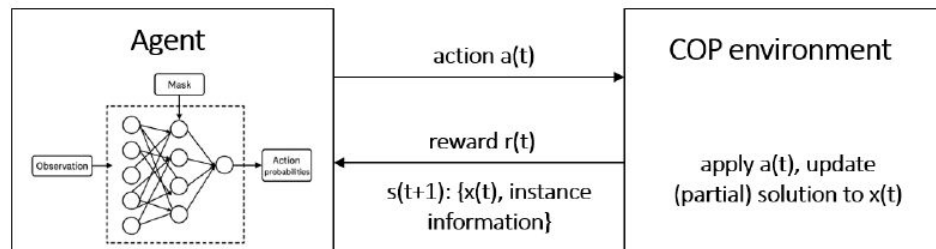
- Starts from an existing complete solution
- Learns to apply local moves to refine it
- Reward = improvement over the best-so-far (incumbent) solution
- Runs for a fixed step budget T , keeps the best solution seen

From COP to sequential decision-making

- COPs maps to sequential decision-making paradigm where an agent can either **learn-to-improve** solutions or **learn-to-construct** solutions.

Learn-to-Construct

- Starts from an empty / trivial state
- Iteratively adds components until a complete solution is formed
- State = static instance info + current partial solution
- Episode ends once the solution is complete



From COP to sequential decision-making

Quality of DRL solution is dependent on problem-formulation, neural network architecture and DRL training process.

From COP to sequential decision-making

Quality of DRL solution is dependent on problem-formulation, neural network architecture and DRL training process.

We highlight four design pillars that need attention:

Instance
encoding
schemes

Markov
decision
process (MDP)
formulation

Neural network
architecture

Reinforcement
learning
algorithms

Design pillars

Markov decision process

- An MDP provides a mathematical description of the environment in which a reinforcement learning agent operates.
- It is formally defined as a tuple $M = (S, A, P, R, \gamma)$.

Design pillars

Markov decision process

State S: represents all possible partial solutions or problem configurations
e.g., TSP: (current city, unvisited set, distance matrix)

Action A(s): set of decisions available to the agent, invalid actions are masked.
e.g., TSP: pick any unvisited city

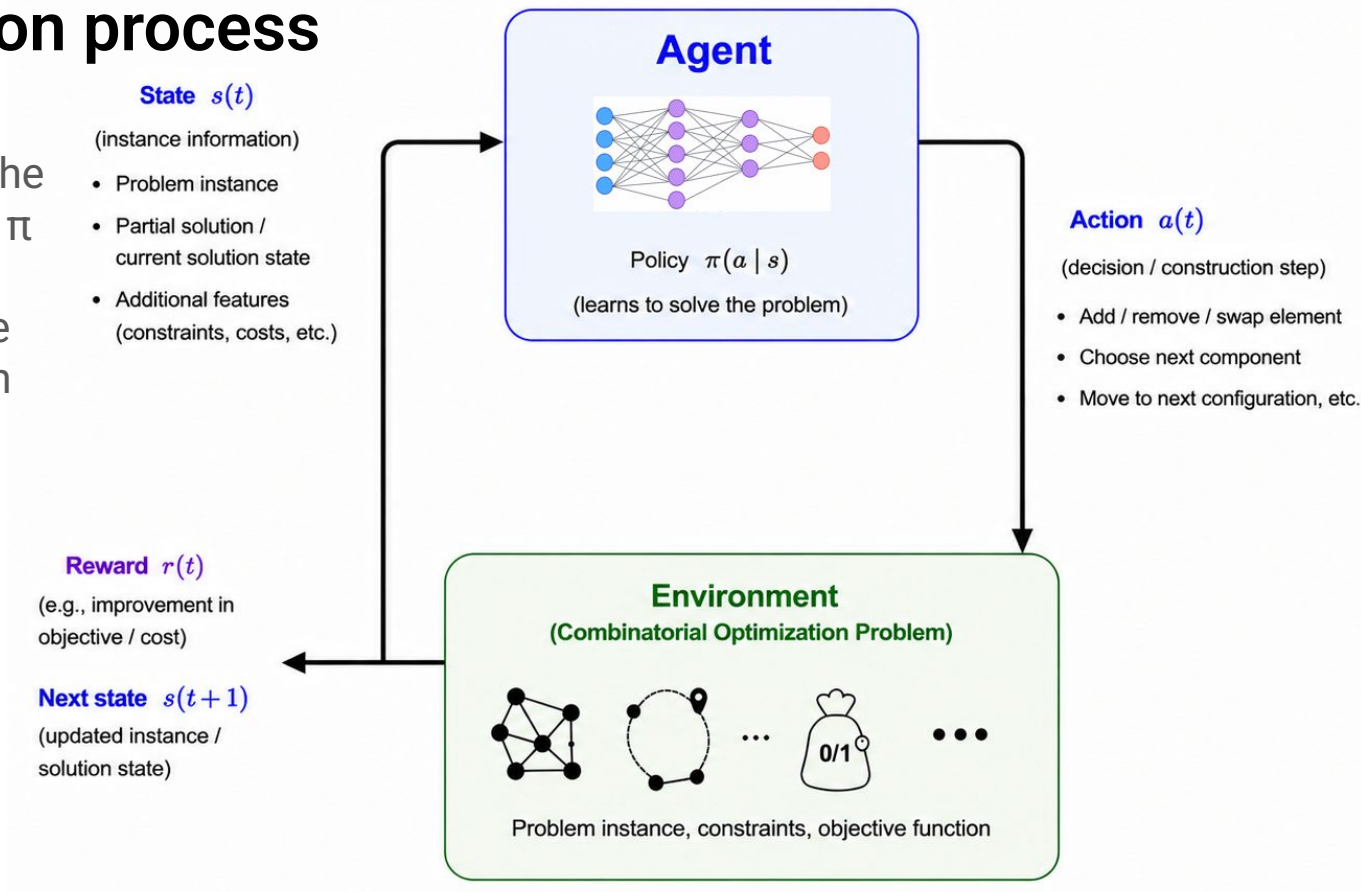
Transition P: probability of moving to next state. In COP, transition are deterministic.
e.g., adding a new city to tour -> new partial solution (next state)

Reward R: provide a feedback on solution quality, can be final cost or improvement.
e.g., tour length at the end or change in tour length.

Discount γ : relative importance of immediate versus future rewards in the cumulative return.
Typically set to 0.99 to give equal weights to sequential construction steps.

Markov decision process

→ The objective of the agent is learn policy π that maximizes the expected cumulative reward (i.e., find high quality solution(s)).



Design pillars

Reinforcement learning algorithms

- Reinforcement learning provides a powerful suite of algorithms designed for this policy search.
- To be specific, a deep RL algorithm updates θ to maximize the expected return

$$J(\theta) = E_{\tau \sim \pi}[R(\tau)]$$

where

$$R(\tau) = r_0 + r_1 + \dots + r_{T-1} = \sum_{t=0}^{T-1} r_t$$

Reinforcement learning algorithms

- Reinforcement learning provides a powerful suite of algorithms designed for this policy search.
- To be specific, a RL algorithm updates θ to maximize the expected return

$$J(\theta) = E_{\tau \sim \pi}[R(\tau)]$$

- RL algorithms can be broadly divided into three categories:



Value-based e.g. DQN

Learn a value function $Q(s,a)$; act by greedily choosing the highest-value action. Policy is implicit.



Policy-based e.g. REINFORCE

Learn $\pi\theta(a|s)$ directly via the policy gradient. The network outputs an action distribution.

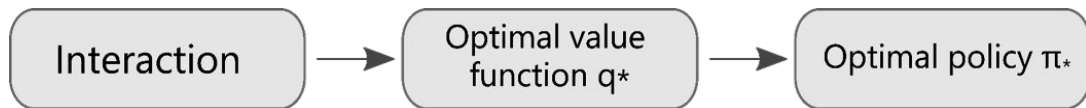


Actor-critic e.g. PPO

An actor (policy) and a critic (value function) trained together. Offers stability and sample-efficiency.

RL algorithms - Value based methods (1/2)

- Learn the action-value function $Q(s,a)$, i.e. the expected return of taking action a in state s , then following the policy π



- Value-based methods are based on TD-learning

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

- Policy is implicitly derived from the estimated value function by acting greedily

$$\pi^*(s) = \arg \max_a Q^*(s, a)$$

RL algorithms - Value based methods (2/2)

- Tabular Q-learning struggle with large state and action spaces.
- Deep Q-Networks (Mnih et al., 2015) replaces the Q-table with neural network (NN)
- It uses experience replay and fixed target network to approximate $Q(s,a) \approx Q_{\theta}(s,a)$, where NN are trained by minimizing the TD error:

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(\underbrace{r + \gamma \max_{a'} Q_{\theta'}(s', a')}_{\text{TD Target}} - \underbrace{Q_{\theta}(s, a)}_{\text{Predicted Q-value}} \right)^2 \right]$$

- Value-based methods are less commonly used for COP, due to requiring enumeration of all actions to find $\text{argmax } Q(s,a)$

RL algorithms - Policy based methods (1/3)

- Learn a parameterized policy $\pi_{\theta}(a|s)$ directly – the network outputs an action distribution.



$$\max_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}}[R(\tau)]$$

Our goal is to find the weights θ that maximize the expected return.

Design pillars

RL algorithms - Policy based methods (2/3)

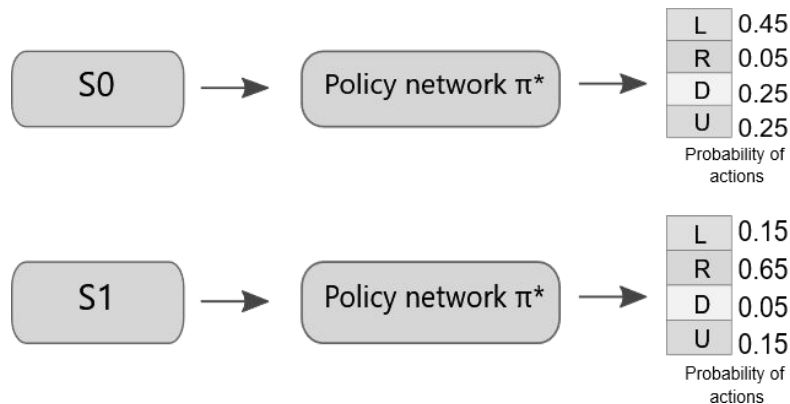
Episode 1: (S0, L), (S1, R), (S2, D), (S3, L)

won

Episode 2: (S0, D), (S4, R), (S10, R)

lost

How do we reinforce the actions that leads to higher rewards?



Amend the network to make it less likely to take the bad decisions/actions

Design pillars

RL algorithms - Policy based methods (3/3)

Reinforce algorithm:

Loop:

Phase 1: Collect episodes with the π .

Calculate the return (sum of rewards).

Phase 2: Tune the weights of the policy network.

If **positive return**, increase the probability of each state, action combination.

If **negative return**, decrease the probability of each state, action combination.

$$\nabla_{\theta} J(\theta) \approx \hat{g} = \frac{1}{m} \sum_{i=1}^m \sum_{t=0} \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} | s_t^{(i)}) R(\tau^{(i)})$$

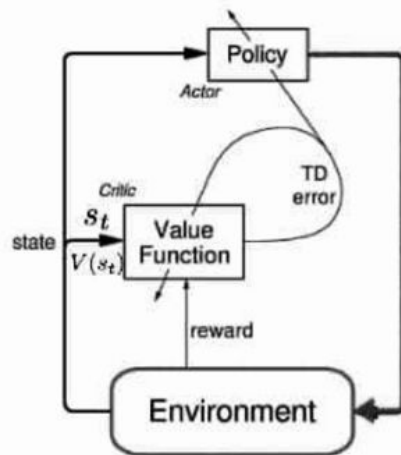
Estimation of the gradient (given we use some trajectories to estimate the gradient)	Scaling factor inversely proportional to the number of trajectories (m)	Probability of the agent to select action at from state s_t given our policy in trajectory (i) Direction of the steepest increase of the (log) probability of selecting action at from state s_t	Cumulative return of i-th trajectory
--	---	---	--------------------------------------

RL algorithms - Actor-critic methods (1/2)

- Two networks, trained together, combining the strengths of value- and policy-based learning.
- Actor: A policy that controls how our agent acts $\pi_{\theta}(s)$
- Critic: A value function to assist the policy update by measuring how good the action taken is

Policy Update: $\nabla_{\theta} U(\theta) \approx \hat{g} = \log \pi_{\theta}(a_t | s_t) R(\tau)$

New Update: $\nabla_{\theta} U(\theta) \approx \hat{g} = \log \pi_{\theta}(a_t | s_t) Q(S_t, A_t)$



- Actor: decides which action to take
- Critic: tells the actor how good its action was and how it should adjust

RL algorithms - Actor-critic methods (2/3)

- We can stabilize learning further by using the advantage function as critic instead of the action value function.
- Advantage function calculates the relative advantage of an action compared to the others possible at a state

Policy Update: $\nabla_{\theta} U(\theta) \approx \hat{g} = \log \pi_{\theta}(a_t | s_t) \cancel{R(\tau)}$

New Update: $\nabla_{\theta} U(\theta) \approx \hat{g} = \log \pi_{\theta}(a_t | s_t) Q(S_t, A_t)$

$$A(s_t, a_t) = (r_{t+1} + \gamma V_{\phi}(s_{t+1})) - V_{\phi}(s_t)$$


RL algorithms - Actor-critic methods (3/3)

- Proximal policy optimisation (PPO) is most widely used actor-critic algorithm in the CO literature.
- Here, we use Clipped surrogate objective function

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right]$$

Where

$r_t(\theta)$ denotes the probability ratio between the current and old policy, ϵ is a hyperparameter to define the clip range.

- Clipping caps how far the new policy can move from the old one to avoid any destructive updates
- PPO offers robust performance and has relatively easy implementation

Routing (VRP/TSP) and scheduling (JSP) as concrete examples

End-to-End DRL for Classical COPs

End-to-End DRL for a COP

For a given COP, developing a DRL model typically involves four key steps:

- designing an **instance encoding scheme** to represent problem instances,
- formulating the solving process as a **Markov decision process**,
- selecting a **neural network architecture** to serve as a parameterized policy updated via reinforcement learning,
- choosing a **reinforcement learning algorithm** to update the policy.

VRP: Instance encoding scheme

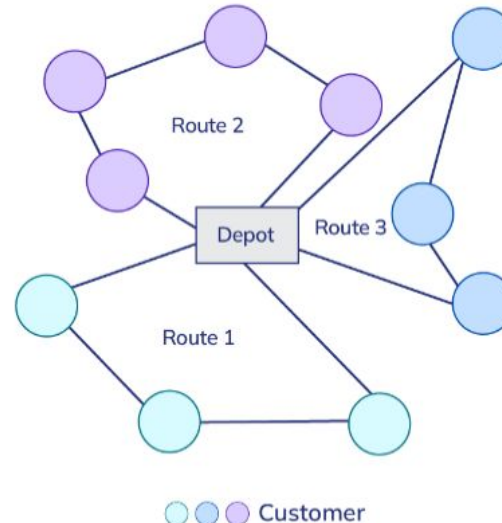
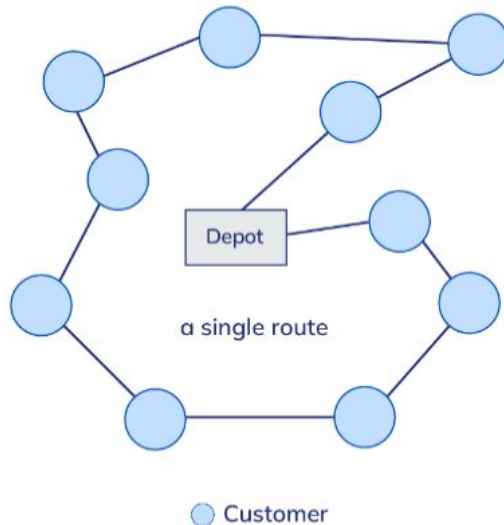
A VRP instance can be defined on a graph $G = \{V, E\}$,

- The node set $V = \{v_0, \dots, v_n\}$
- The edge set $E = \{(v_i, v_j) | v_i, v_j \in V, v_i \neq v_j\}$. v_0 denotes the depot (e.g., in CVRP).
- The nodes carry spatial coordinates x_i , as well as additional attributes depending on the problem type, e.g., demand d_i in CVRP.
- A feasible solution to a VRP is a tour τ , an ordered sequence of nodes. With a cost function $\mu(\cdot)$ defined on tours, solving the VRP is to identify the best tour among all feasible ones.

VRP: Instance encoding scheme

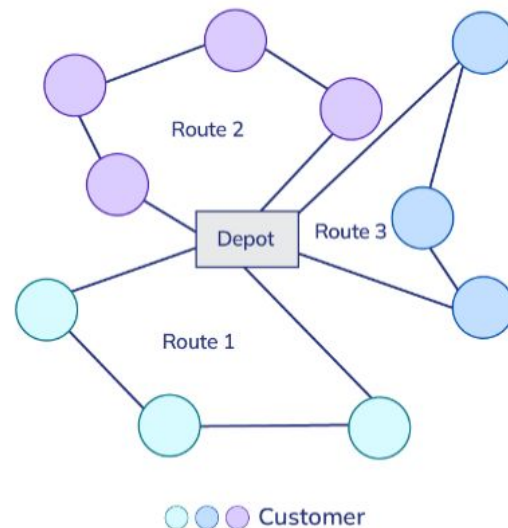
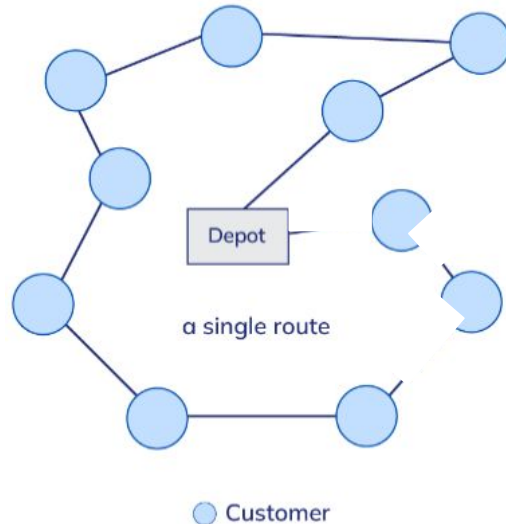
Feasibility means:

- For TSP, a feasible TSP tour must visit every node in the graph exactly once
- For CVRP, a solution is feasible if each customer node is visited exactly once and the demand served on any vehicle's tour does not exceed its capacity



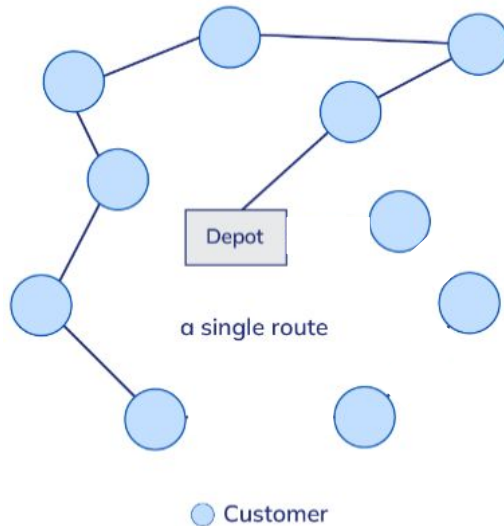
VRP: Markov decision process

State: The state $s_t \in S$ at time t encapsulates the **static instance information** for the customer and depot node, e.g., coordinates, demands; as well as the **dynamic information** during the route construction, e.g., the remaining vehicle capacity and the current partial solution $\{a_1, \dots, a_{t-1}\}$.



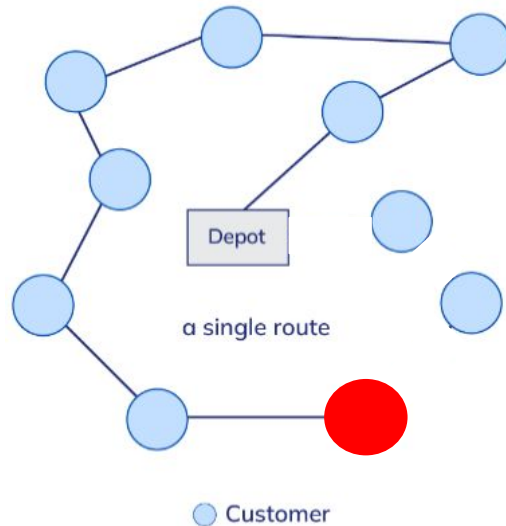
VRP: Markov decision process

Action: The action consists of selecting a valid node $a_t \in V_t$ from the node set $V_t \subseteq V$. To prevent choices that would render the solution infeasible, invalid nodes should be masked, leading to the current action space V_t . For instance, in the TSP, the visited nodes are masked to avoid subtours.



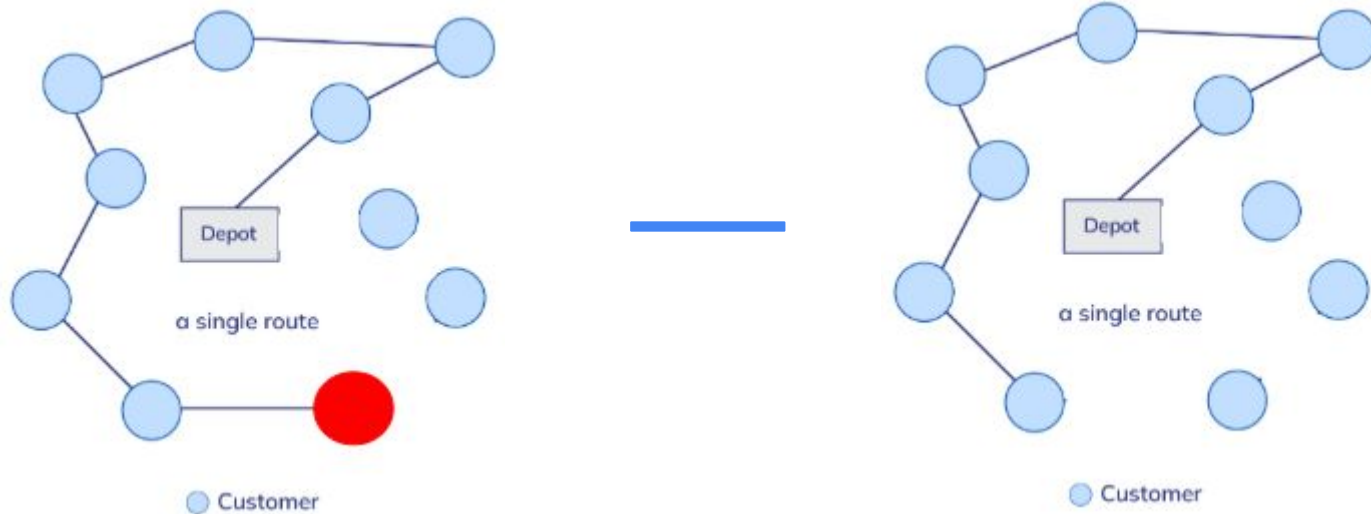
VRP: Markov decision process

Transition: In the state transition, the action a_t (i.e., the selected node) is added to the partial solution, which is updated to $\{a_1, \dots, a_{t-1}, a_t\}$. The other dynamic information in the state is updated as well..



VRP: Markov decision process

Reward: In VRPs, the objective is typically to minimize tour length. The total reward of a constructed solution (i.e., a sequence of actions) can be defined as the negative of the total tour length.

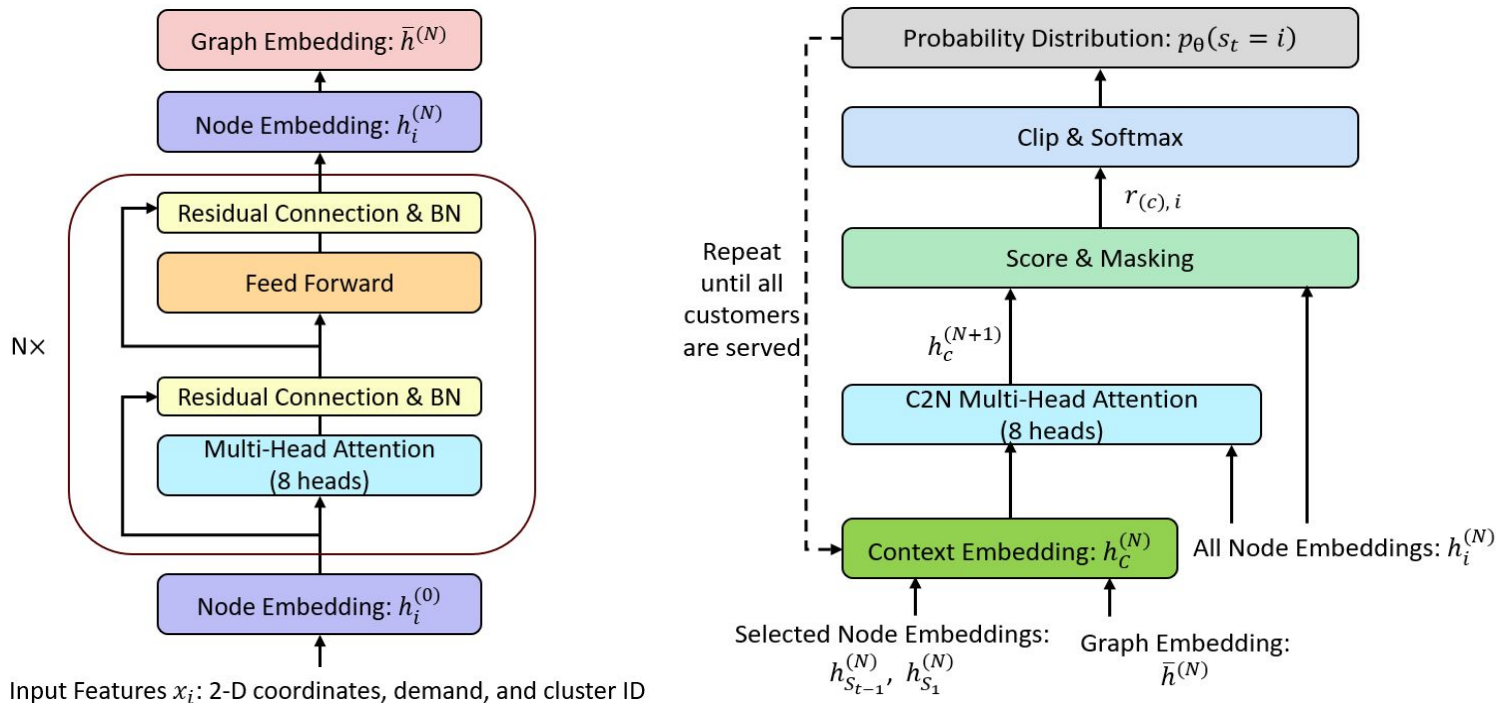


VRP: Markov decision process

- **Policy:** Given each state s_t at each step t , the policy π selects an action a_t (i.e., a node) by learning a probability distribution over nodes.
- The probabilities of infeasible actions are set to 0.
- The MDP episode terminates once a valid tour is formed.
- This construction trajectory follows a probability chain $P(s_T | s_0) = \pi(a_0 | s_0) \times \dots \times \pi(a_t | s_t)$, where T represents the number of nodes in the formed solution.

VRP: Transformer as policy network

Transformer models may vary slightly due to differences in raw features and constraints.



VRP: Reinforcement learning algorithm

REINFORCE algorithm for VRPs. Let θ be the policy parameters and $R(\tau)$ the reward of sampled trajectory τ . Given the probability chain $P(s_T | s_0) = \pi(a_0 | s_0) \times \dots \times \pi(a_t | s_t)$, In standard REINFORCE (Williams, 1992), the gradient is calculated as:

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[(R(\tau) - b(\mathcal{G})) \nabla_{\theta} \log P_{\theta}(\tau | \mathcal{G}) \right],$$

- where $b(\mathcal{G})$ is a baseline function designed to reduce gradient variance. As shown in Kwon et al. (2020), a widely used baseline is the average of a batch of sampled solutions.

Williams, R.J., 1992. Simple statistical gradient-following algorithms for connectionist reinforcement learning. Machine learning 8, 229–256.

Kwon, Y.D., Choo, J., Kim, B., Yoon, I., Gwon, Y., Min, S., 2020. Pomo: Policy optimization with multiple optima for reinforcement learning. Advances in Neural Information Processing Systems 33, 21188–21198.

JSP: Definition

A JSP instance comprises a set of jobs $J = \{J_1, \dots, J_{|J|}\}$ and a set of machines $M = \{M_1, \dots, M_{|M|}\}$.

- Each job $J_i \in J$ contains an operation set $O_i = \{O_{i1}, O_{i2}, \dots\}$ that must be processed in a specific order $O_{i1} \rightarrow O_{i2} \rightarrow \dots$
- Each O_{ij} is featured by a processing time p_{ij} .
- Each machine only handles a single operation at any given time.
- The goal in a JSP is to assign a start time S_{ij} to every operation O_{ij} to minimize the overall completion time (makespan): $C_{\max} = \max_{i,j} \{S_{ij} + p_{ij}\}$

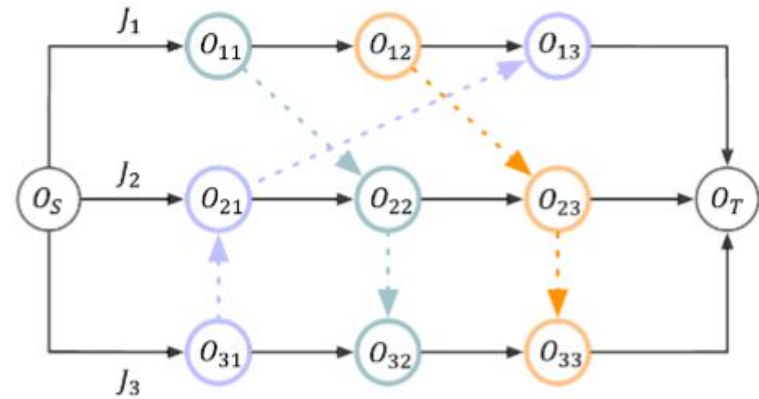
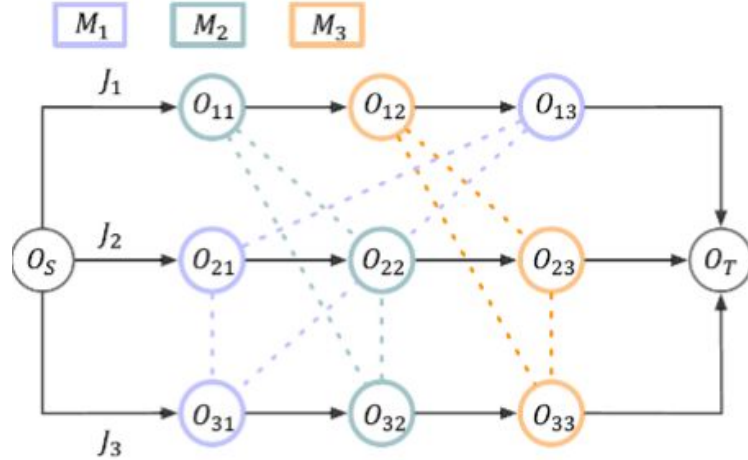
JSP: Instance encoding scheme

JSP instance can be represented as a disjunctive graph $G = (O, C, D)$.

- The set of operations is given by $O = \{O_{ij} \mid \forall i, j\} \cup \{O_S, O_T\}$, where O_S and O_T are dummy operations of zero processing time.
- The arc set C comprises **directed edges (conjunctions)** that capture precedence constraints among operations of the same job, while D comprises **undirected edges (disjunctions)** that connect operations competing for the same machine.
- Solving a JSP then amounts to orienting each disjunctive edge so that the resulting graph is a directed acyclic graph (DAG).

JSP: Instance encoding scheme

Disjunctive graph representation of a JSP instance of 3 jobs and 3 machines.



JSP: Markov decision process

- **State:** The state $s_t \in S$ consists of the problem instance together with the partial schedule constructed thus far, which is represented by the graph.
- **Action:** Given the current state, the action $a_t \in A_t$ is defined as an operation that is dispatched to the corresponding machine. The maximum size of action space is $|J|$.
- **Transition:** The transition updates the state (i.e., the graph) based on the selected action (i.e., the operation). For example, this may involve orienting a disjunctive edge.
- **Reward:** r_t is defined as the difference in estimated makespan between two consecutive states.
- **Policy:** Given each state s_t at each step t , the policy π selects an action a_t (i.e., an operation node) by learning a distribution over the valid nodes.

JSP: GNN + MLP

GNN is used to get the operation node embeddings. For example, the commonly used GIN (Xu, 2019) performs the computation below:

$$h_{v_i}^{(k)} = \text{MLP}^{(k)} \left((1 + \epsilon^{(k)})h_{v_i}^{(k-1)} + \sum_{v_j \in N(v_i)} h_{v_j}^{(k-1)} \right)$$

MLP is used to obtain a scalar score $\text{MLP}_{\theta}([h_{a_t}, h_{v_g}])$ for each a_t , where $[.]$ means concatenation. Then, a softmax function is applied to output a distribution for node selection.

Xu, K., Hu, W., Leskovec, J., Jegelka, S., 2019. How powerful are graph neural networks?, in: International Conference on Learning Representations (ICLR).

VRP: PPO algorithm

$$L^{CLIP}(\theta) = \min \left(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t \right)$$

- $r_t(\theta)$ is the probability ratio of taking actions under the new policy to take the same actions under the old policy $\frac{\pi_{\theta}(a_{n,t} | s_{n,t})}{\pi_{\theta_{old}}(a_{n,t} | s_{n,t})}$
- A_t is the estimated advantage of taking action a_t at time step t , representing the expected reward difference between the current policy and the target $A_t = P_t \sum \gamma_t r_t - v_{\phi}(s_t)$
- The clipping balances the exploitation of the improvement and avoids making too big of a change in a single iteration.

Extending to variants with multiple objectives and uncertainties

Beyond Classical VRP and JSP

Multi-objective optimization

Multi-Objective Combinatorial Optimization (MOCO) can be formulated as: $\min_{x \in X} f(x) = (f_1(x), f_2(x), \dots, f_N(x))$,

- X denotes the set of feasible solutions
- N is the number of objective functions to be optimized
- each $f_i(x)$ represents an objective to be minimized

Multi-objective optimization

A common approach to deal with a MOCO problem is as follows. It is first decomposed into K single-objective subproblems according to respective preferences $\lambda_1, \lambda_2, \dots, \lambda_K$. Each preference satisfies $\lambda_i = (\lambda_{i1}, \dots, \lambda_{iN})^\top$, $\forall \lambda_{ij} \geq 0$ and $\lambda_{i1} + \dots + \lambda_{iN} = 1$. The commonly used decomposition approaches are weighted-sum approach and Tchebycheff approach.

- The i th single-objective subproblem is defined by the following scalarized objective function:

$$\min \quad g_w(\tau|\lambda_i) = \sum_{j=1}^N \lambda_i^j f_j(\tau), \quad \tau \in \mathcal{X}$$

- Tchebycheff approach decomposes an MOCO problem by minimizing the maximal distance between objective values and the ideal reference point such that

$$\min \quad g_t(\tau|\lambda_i, z^*) = \max_{1 \leq j \leq N} \left\{ \lambda_i^j |f_j(\tau) - z_j^*| \right\}, \quad \tau \in \mathcal{X}$$

REINFORCE Algorithm for MOCO

Given an MOCO problem, we define the reward for each subproblem by the decomposition approaches, taking Tchebycheff approach as an example:

$$R(\tau|s, \lambda_b) = - \max_{1 \leq i \leq N} \{ \lambda_b^i |f_i(\tau) - (z_i^* - \varepsilon)| \}$$

- The REINFORCE (Williams, 1992) algorithm calculates the gradient to maximize the expected total reward by gradients:

$$\nabla J(\theta) = E_{\tau \sim p_\theta, s \sim \mathcal{S}_I, \lambda \sim \mathcal{S}_\lambda} [(R(\tau|s, \lambda) - \tilde{R}(s, \lambda)) \nabla_\theta \log p_\theta(\tau|s, \lambda)]$$

- Given B instances and N sampled solutions, we calculate the approximate gradient:

$$\nabla J(\theta) \simeq \frac{1}{BN} \sum_{b=1}^B \sum_{j=1}^N (R(\tau_b^j|s_b, \lambda_b) - \tilde{R}(s_b, \lambda_b)) \nabla_\theta \log p_\theta(\tau_b^j|s_b, \lambda_b)$$

REINFORCE Algorithm for MOCO

Algorithm 3 REINFORCE for MOCO

Input: Instance distribution $\mathcal{S}_I$, preference distribution $\mathcal{S}_\lambda$, number of training epochs E , number of training instances per epoch $\mathbb{D}$, batch size B , number of solutions $\mathcal{N}$.

Output: The trained policy network (i.e. p_θ).

```
1: Initialize policy network parameters  $\theta$ .
2: for  $e = 1$  to  $E$  do
3:    $D = 0$ ,
4:   while  $D < \mathbb{D}$  do
5:      $\tilde{B} = \min(\mathbb{D} - D, B)$ ,
6:      $\lambda_b \sim \text{SAMPLEPREFERENCE}(\mathcal{S}_\lambda), \forall b \in \{1, \dots, \tilde{B}\}$ ,
7:      $s_b \sim \text{SAMPLEINSTANCE}(\mathcal{S}_I), \forall b \in \{1, \dots, \tilde{B}\}$ ,
8:      $\tau_b^j \sim \text{SAMPLESOLUTION}(p_\theta(\cdot | s_b, \lambda_b)), \forall j \in \{1, \dots, \mathcal{N}\}, \forall b \in \{1, \dots, \tilde{B}\}$ ,
9:     Calculate gradient  $\nabla J(\theta)$  according to Eq. (12),
10:     $\theta \leftarrow \text{ADAM}(\theta, \nabla J(\theta))$ ,
11:     $D = D + \tilde{B}$ ,
12:   end while
13: end for
```

Stochastic COPs

Stochastic COPs refer to problems in which some parameters are not fixed, but vary across different scenarios.

For example, given the instance encoding for JSP, each scenario represents the realized values for the processing time.

By applying scenarios to the instance encoding, we can obtain **a set of graph representations** for JSPs with different scenarios.

How do we address these graph representations?

MDP for stochastic COPs

Given the deterministic COP instance, the states should capture the features on graphs of deterministic instance and stochastic scenarios.

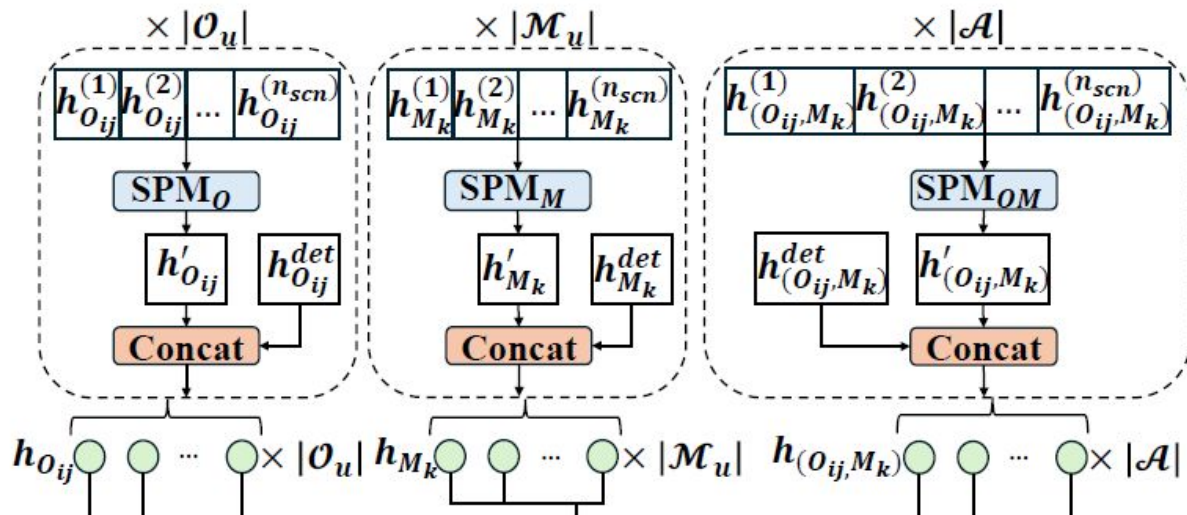
- The state at step t can be defined as $s_t = \{s^{\text{det}}(t), S^{\text{stoch}}(t)\}$, where $s^{\text{det}}(t)$ denotes the state of the deterministic instance, and $S^{\text{stoch}}(t)$ denotes the states of scenarios $\{s^1(t), \dots, s^L(t)\}$
- The state transition should be applied to the states of deterministic instance and all stochastic scenarios
- The reward is $r_t = f(C_t) - f(C_{t+1})$, where C_t is the set of rewards for scenarios and f is a function to aggregate the rewards, e.g., average reward.

Policy network for stochastic COPs

- The most straightforward approach is to process the scenario graphs with the same GNN and concatenate or add their embeddings (e.g., operation and machine embeddings in JSP).
- Statistics of features are put on graph, simplifying the GNN processing still on a single graph with average, variance, ... information.

Policy network for stochastic COPs

The attention can be used to include the scenarios embeddings, e.g., SPM (Smit et al., 2025).



Smit, I.G., Wu, Y., Troubil, P., Zhang, Y., Nuijten, W.P., 2025a. Neural combinatorial optimization for stochastic flexible job shop scheduling problems, in: Proceedings of the AAAI Conference on Artificial Intelligence, pp. 26678–26687.

Agenda

Part 2 - Hands-on exercises

- DRL for real-world problems
- Open challenges and takeaways
- Code walkthrough - Job Shop Scheduling Problem (JSP)
- Code walkthrough - Capacitated Vehicle Routing Problem (CVRP)

DRL for real-world optimisation problems



KONINKLIJKE
LANDSCHE MAATSCHAPPIJ
DER WETENSCHAPPEN



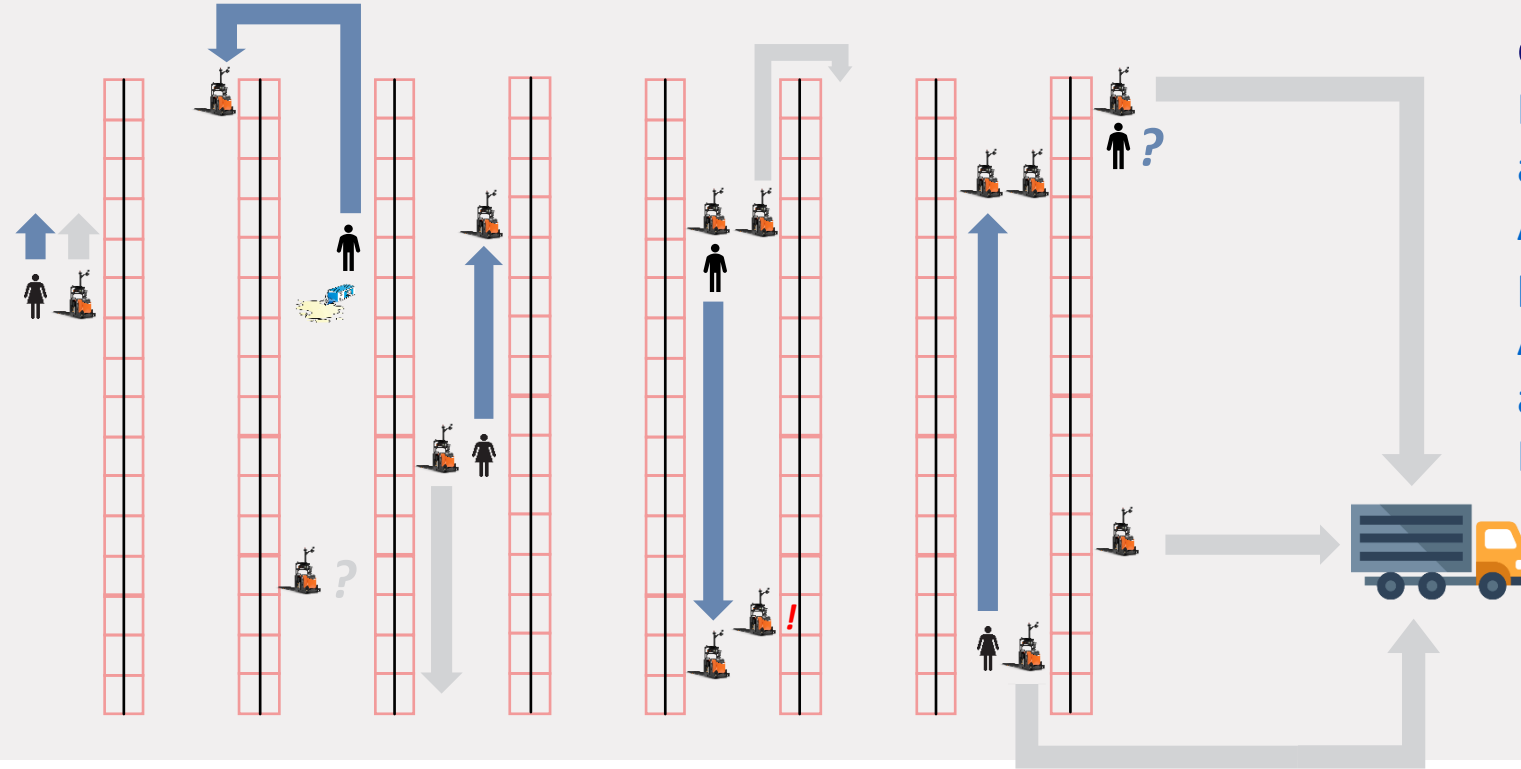
VANDERLANDE

Learning Efficient and Fair Policies for Uncertainty-Aware Collaborative Human-Robot Order Picking



Order Picking: crucial component of warehouse operation

a sequential decision problem



Collaborative
order picking:
Pickruns are
assigned to AMRs;
AMR moving to a
picking location;
A human picker is
assigned to AMR;
Repeat

Optimization problem

Build a picker optimizer, that *allocates human pickers to AMRs, such that the total picking time is minimized (i.e., max pick rate)*

Solution requirements

- Online (real-time) allocation/decision
- Handling uncertainties, randomness, and congestion
- Working well for different scenarios (e.g., picker/AMR numbers, warehouse layouts and sizes)

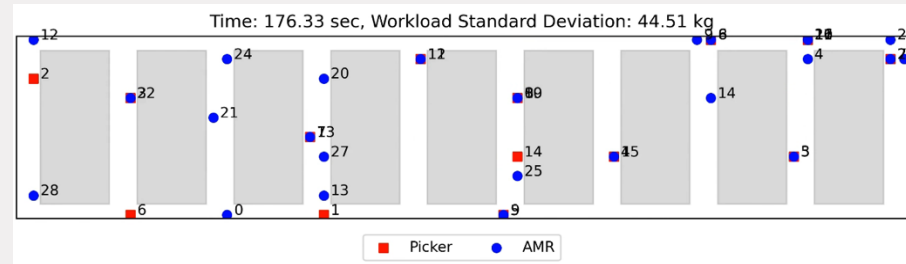
- New warehouse setup: no real-world operation data
 - However
 - Operation processes are known
 - Information of orders, locations of items can be obtained from other warehouses
- > We can build a rather realistic simulation model

Environment: simulation model

Representing interactive processes between pickers, AMRs, and the picker optimizer.

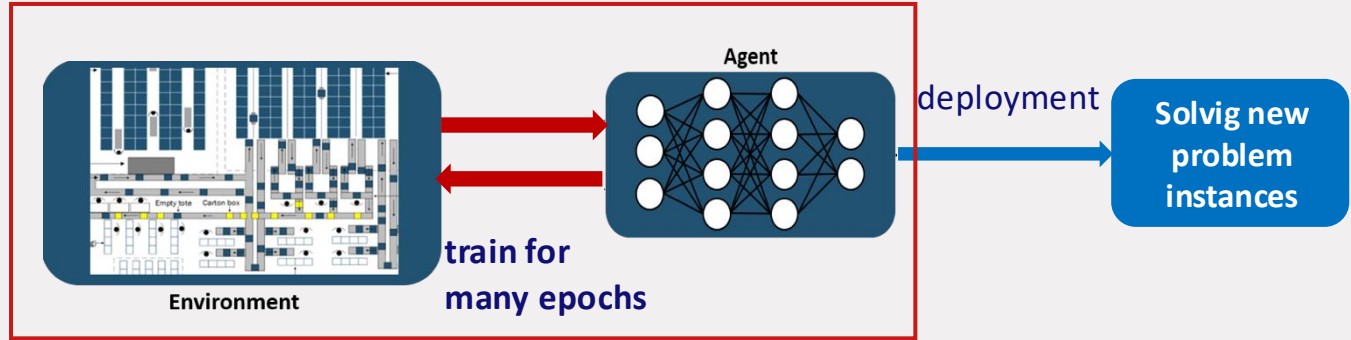
Considering uncertainties

- stochastic picker/AMR speeds and picking times
- picking disruptions
- congestion causing delays for the AMRs related to overtaking procedures



An End-to-End online DRL Picker Optimizer

The simulator:
as environment



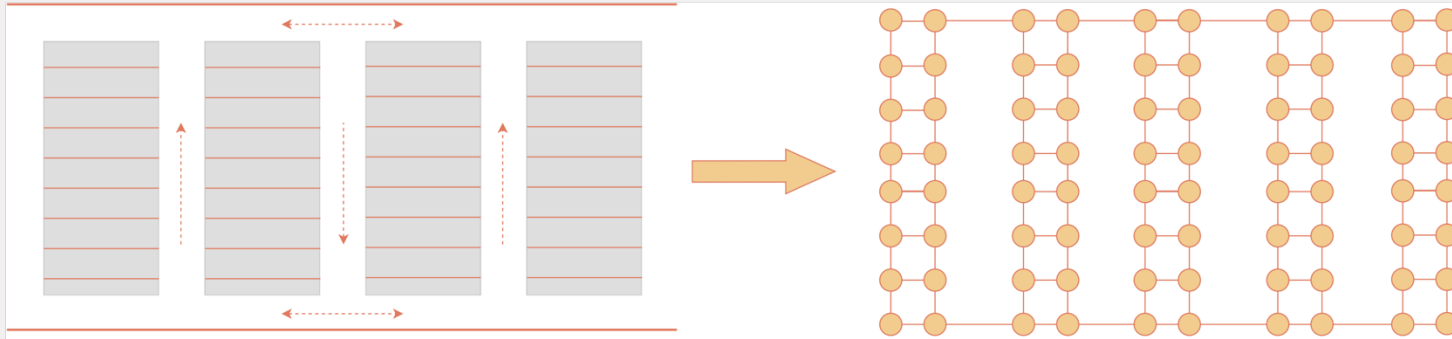
Steps:

1. Instance encoding scheme
2. Markov decision process (MDP) formulation
3. Neural network architecture
4. Reinforcement learning algorithm

Step 1 Instance encoding: spatial input representation

Instance as a graph

- Nodes: locations in warehouse
- Edges: how to travel between locations



Step 2 MDP: State Representation

Node features capture information for each node/location

1. Controlled picker
2. AMR information
3. Picker information
4. Node location information
5. Node neighborhood information

Step 2 MDP: Action space; Reward Function

Action Space:

- New picker destination
- Truncated action space: current or next AMR destination where no other picker is going
- Reward function: Penalty on time that passes

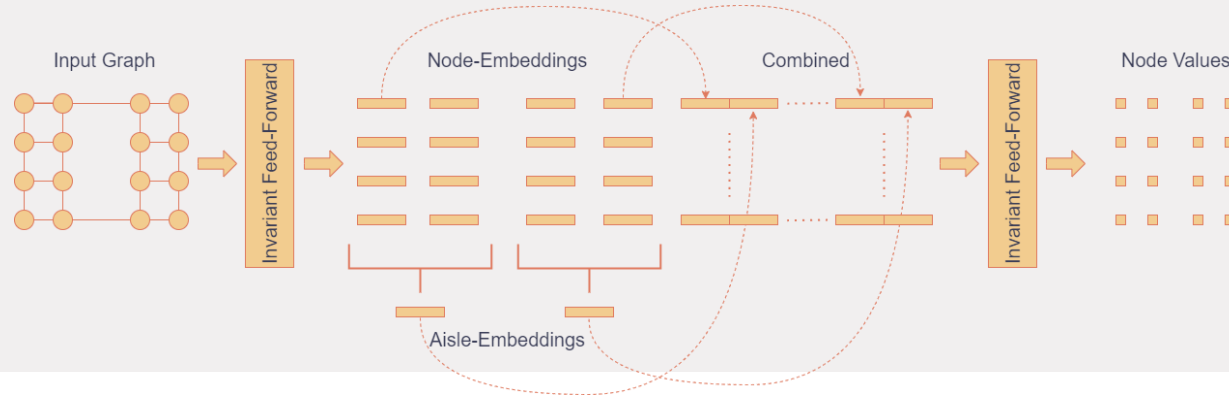
$$R_t^{\text{efficiency}} = \tau_{t-1} - \tau_t$$

Step 3 Agent NN architecture design

Network must handle graph space and extract spatial information

~~Regular graph neural networks~~ → Message passing

New architecture: We adapt Invariant Feed-Forward network, with aisle-embedding: capture spatial relations in warehouse, size agnostic



Step 4: RL Learning Algorithm

Proximal Policy Optimization (PPO) (Schulman et al., 2017)

- Policy-based method: policy network outputs the actions

Actor-critic

- Actor network: suggests actions
- Critic network: estimates the advantage function (how good is the selected action)

Experimental settings

- Performance measure: picking time

Multiple warehouse types

Warehouse Type	Aisles	Aisle Depth	Locations	Pickers	AMRs	Picks
XS	7	7	98	4	7	≤ 100
S	10	10	200	10	25	5000
M	15	15	450	20	50	7500
L	25	25	1250	30	90	7500
XL	35	40	2800	60	180	15000

Benchmark algorithms

- VI benchmark: rule-based method considers the distance of potential picks and also tries to spread the pickers across aisles
- Greedy : assign a picker to the closest available location where an AMR is going, and no other picker is already going.
- MILP: assuming no uncertainties/randomness, no congestions, in XS environment

min C , subject to

$$\begin{aligned}
 \sum_{k \in \mathcal{K}} A_{i,k} &= 1 & \forall i \in \mathcal{N} & \quad (2.3) \\
 A_{i,k} - A_{i',k} &\leq 1 - (U_{i,i'} + U_{i',i}) & \forall i, i' \in \mathcal{N}, i \neq i', k \in \mathcal{K} & \quad (2.4) \\
 A_{i,k} + A_{i',k} &\leq 1 + (U_{i,i'} + U_{i',i}) & \forall i, i' \in \mathcal{N}, i \neq i', k \in \mathcal{K} & \quad (2.5) \\
 B_{i,k}^K &\geq \tau_{i,k}^{0,K} - M \cdot (1 - A_{i,k}) & \forall i \in \mathcal{N}, k \in \mathcal{K} & \quad (2.6) \\
 \sum_{k \in \mathcal{K}} B_{i,k}^K &\geq \sum_{k \in \mathcal{K}} F_{i',k}^K + \tau_{i',i}^K \cdot U_{i',i} - M \cdot (1 - U_{i',i}) & \forall i, i' \in \mathcal{N}, i \neq i' & \quad (2.7) \\
 F_{i,k}^R &\geq F_{i,r}^R - M \cdot (2 - A_{i,k} - a_{i,r}^R) & \forall i \in \mathcal{N}, k \in \mathcal{K}, r \in \mathcal{R} & \quad (2.8) \\
 \sum_{r \in \mathcal{R}} B_{i,r}^R &\geq \left(\sum_{r \in \mathcal{R}} F_{i',r}^R + \tau_{i',i}^R \right) \cdot u_{i',i}^R & \forall i, i' \in \mathcal{N}, i \neq i' & \quad (2.9) \\
 B_{i,r}^R &\geq \tau_{i,r}^{0,R} \cdot a_{i,r}^R & \forall i \in \mathcal{N}, r \in \mathcal{R} & \quad (2.10) \\
 B_{i,r}^R &\geq B_{i,k}^R - M \cdot (2 - A_{i,k} - a_{i,r}^R) & \forall i \in \mathcal{N}, r \in \mathcal{R}, k \in \mathcal{K} & \quad (2.11) \\
 F_{i,r}^R &= B_{i,r}^R + \eta_i^L \cdot a_{i,r}^R & \forall i \in \mathcal{N}, r \in \mathcal{R}, & \quad (2.12)
 \end{aligned}$$

DO NOT READ!
(is incomplete)

Experiment: performance on deterministic, XS instances

Compared to MILP solver
(Gurobi)

*Small instances without
randomness*

Each AMR does 1 pickrun.

(b) Instances of type XS without diverse starting.

Instance	DRL	Greedy	VI Benchmark	MILP	MILP gap (%)
1	244	262	355	244	28.2
2	249	253	297	271	28.1
3	265	272	299	267	29.3
4	240	257	269	245	22.8
5	251	255	277	260	30.9

Gurobi: gaps *after 20 hours*

DRL: solves in milliseconds

Experiment: performance with various sizes

Values indicate the average picking time over 100 evaluation episodes

Warehouse	DRL		Greedy		VI Benchmark
	Picking Time	%	Picking Time	%	Picking Time
S	8586 ± 62	14.9	10619 ± 59	−5.3	10087 ± 58
M	8425 ± 46	21.0	11023 ± 58	−3.3	10669 ± 41
L	6540 ± 37	31.7	9823 ± 33	−2.7	9569 ± 61
XL	9010 ± 21	33.6	13972 ± 44	−3.0	13570 ± 72

Experiment: Picker/AMR Transferability

(a) Warehouse type S.

Pickers/AMRs	DRL		Greedy		VI Benchmark
	Picking Time	%	Picking Time	%	Picking Time
7/15	12825 ± 83	17.1	15166 ± 74	2.0	15472 ± 87
10/20	9206 ± 51	19.4	11274 ± 69	1.3	11420 ± 56
10/30	8221 ± 54	13.0	10283 ± 60	-8.8	9447 ± 52
15/25	6737 ± 42	21.5	7994 ± 40	6.9	8583 ± 36
15/30	5930 ± 34	24.7	7804 ± 55	1.0	7879 ± 46
15/35	5938 ± 35	16.6	7550 ± 44	-6.0	7121 ± 38

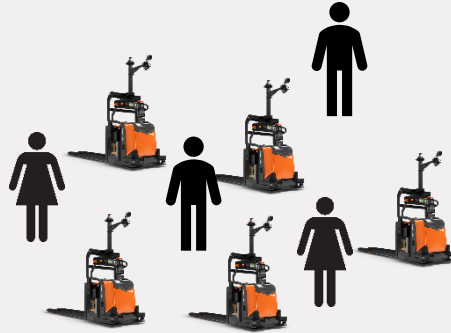
(d) Warehouse type XL.

Pickers/AMRs	DRL		Greedy		VI Benchmark
	Picking Time	%	Picking Time	%	Picking Time
50/120	12028 ± 23	40.2	16816 ± 32	16.5	20142 ± 112
60/140	10150 ± 20	40.7	14312 ± 27	16.4	17118 ± 101
60/200	9009 ± 44	35.6	14293 ± 88	-2.2	13979 ± 87
80/180	8106 ± 77	38.9	11343 ± 30	14.6	13275 ± 83
80/200	8011 ± 59	36.8	11765 ± 52	6.4	12571 ± 91
80/220	6947 ± 19	41.5	10799 ± 40	9.1	11877 ± 84

Trained with:

- 10/25 for S
- 60/180 for XL

The trained policies are directly applied to different scenarios, i.e., different numbers of pickers and AMRs and their ratios



Humans are not robots
where is fairness?

Extending to multi-objective DRL

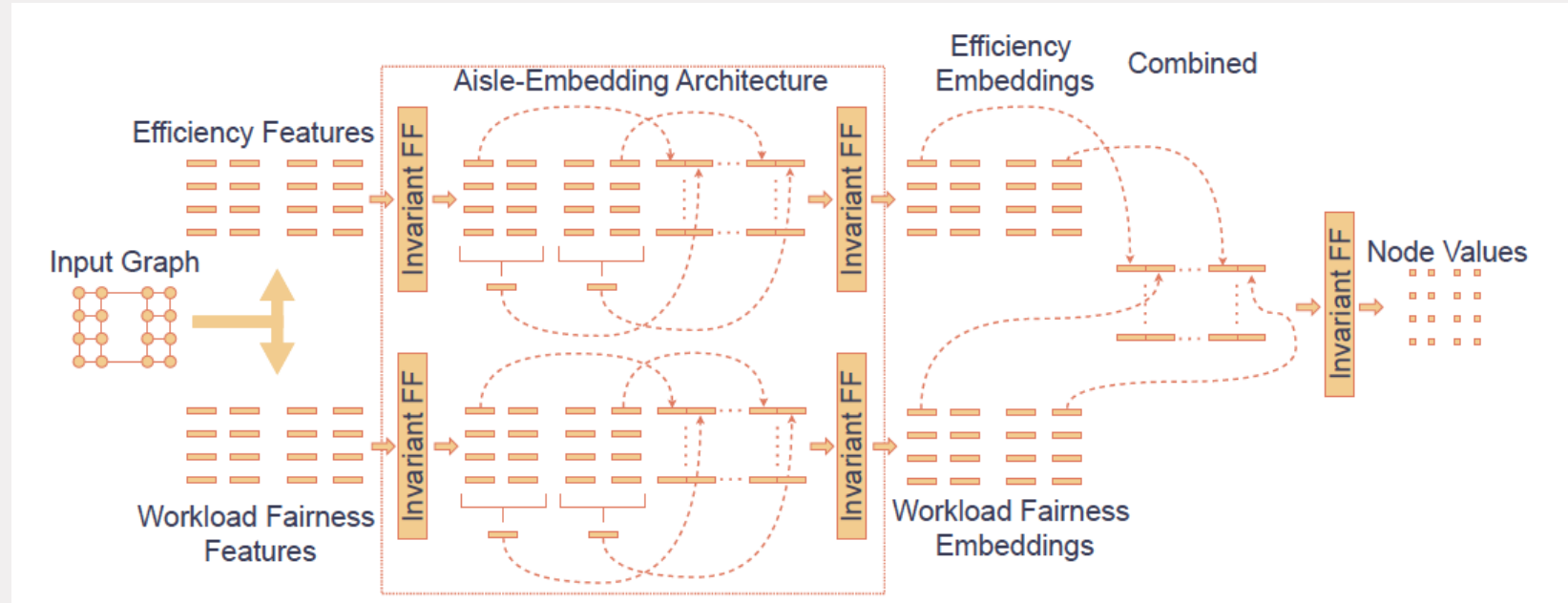
Update MDP:

- **Fairness:** Minimize standard deviation of carried product masses
- Add to state space workload **fairness features**
 - Node-specific information, distributional information
- Add to reward function
 - **penalty on increase in standard deviation of fairness**

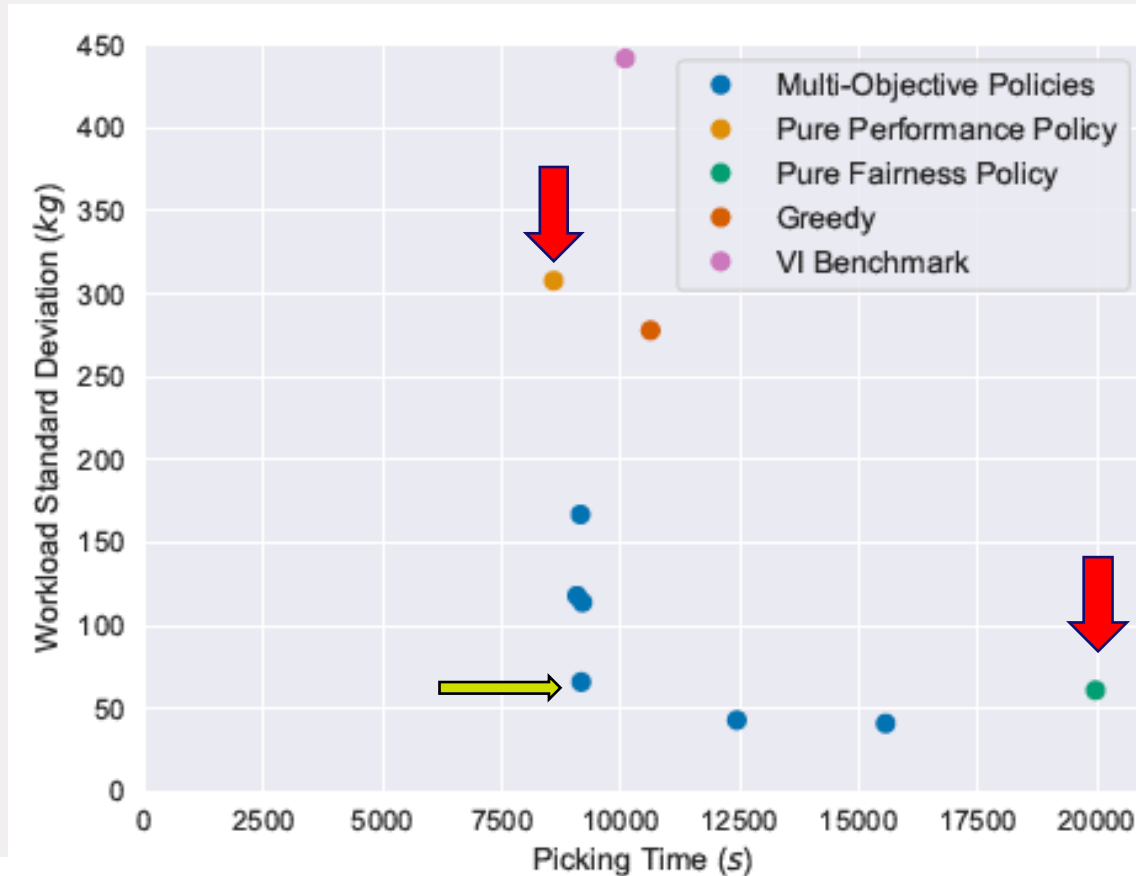
Update learning algorithm

- Adapted from the prediction-guided MORL (Xu et al., 2020)
 - A meta-policy approach, to present a non-dominated set showing the trade-offs

Uplate architecture



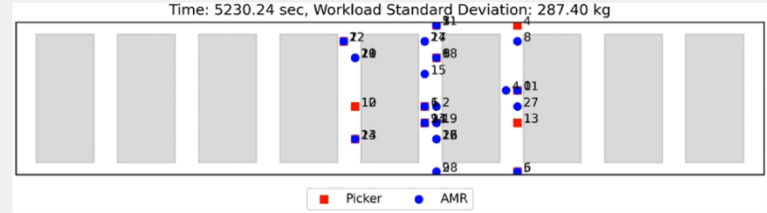
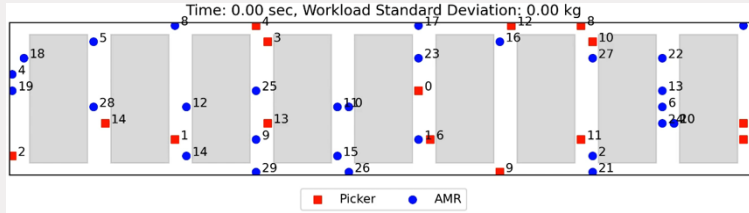
Experiment: *trade-off between fairness and pick rate*



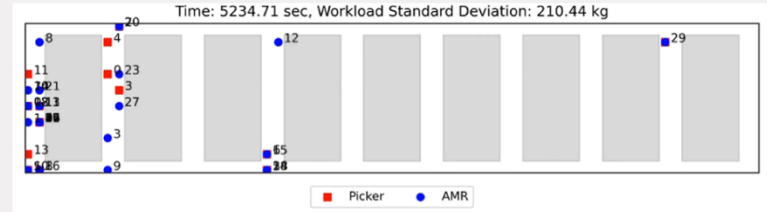
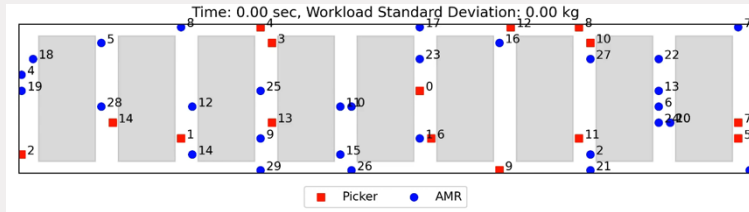
This MORL policy:
by sacrificing just **6.7%** of pick rate efficiency, it decreases the workload standard deviation by **78.6%**

Inspecting Policy Behavior

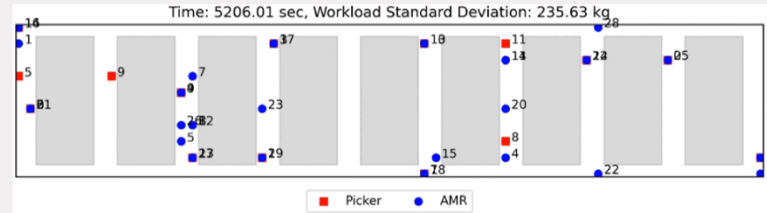
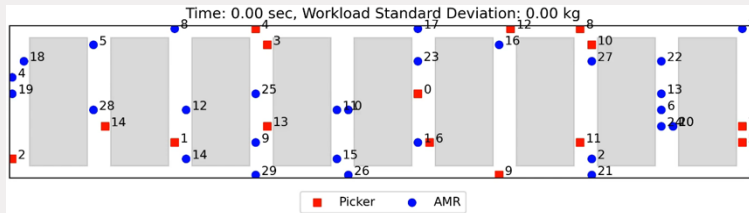
VI Benchmark



Greedy



DRL



E2E online RL for collaborative picking

- Efficiency improvements +- 40%
- Good trade-off between efficiency and fairness
 - Simultaneous improvement of picking times and workload fairness
 - For large warehouse policy with 23.6% efficiency and 92% fairness improvements
- Good transferability:
once trained, can be directly applied to various scenarios and sized warehouses

DRL for real-world COPs

Replace hand-crafted heuristics; fast and adaptive decisions; handle uncertainties

- Learning to optimize human-robot picking & to batching orders in warehouses; to shunt trains and railcar itinerary; to optimize ordering policy for platelet inventory problem in blood supply chain; to plan maintenances

See survey papers:

- *Chung, K., Lee, C., Tsang, Y., 2025. Neural combinatorial optimization with reinforcement learning in industrial engineering: a survey.*
- *Esteso, A., Peidro, D., Mula, J., Díaz-Madroñero, M., 2023. Reinforcement learning applied to production planning and control.*
- *Lai, X., Yang, Z., Xie, J., Liu, Y., 2024. Reinforcement learning in transportation research: Frontiers and future directions.*
- *Ngwu, C., Liu, Y., Wu, R., 2025. Reinforcement learning in dynamic job shop scheduling*

Challenges that limit real-world adoptions!

Open Challenges & Takeaways

Open challenges

Current benchmarking practices

- Classical benchmarks weren't built to evaluate the DRL's strengths.
- TSPLIB, Taillard, Hurink instances are static and deterministic that designed for classical solvers.
- Real-world problems has dynamic/stochastic settings, where DRL perform better
- Academic papers use custom-generated instances that vary in size, distribution, and difficulty, which makes it very difficult to perform cross-paper comparison
- There are also no standardized evaluation protocol, different metrics, baselines, and optimality-gap definitions across studies

Recently, there are domain-specific suites (scheduling: Reijnen et al., 2023 · routing: Kool et al., 2023 / AI4TSP) and a cross-domain benchmark, RL4CO (Berto et al., 2025) are published, but community-wide adoption remain limited.

Open challenges

Adaptability to new settings

- Real-world systems never stop changing and trained policies often struggle to don't keep up.
- There is problem of domain shift where
 - demand pattern can change,
 - new constraints can appear, or
 - new problem variant may emerge.

LLM-based solvers are being explored for few-shot generalization — e.g. an end-to-end LLM COP solver (Jiang et al., 2026b)

Open challenges

Highly constrained COPs

- Action masking assumes feasibility can be maintained step by step which is true only for locally decomposable constraints
- Many real COPs have global constraints — e.g. VRP synchronization across vehicles — verifiable only once the full solution exists
- Current workarounds include reward penalties or post-hoc repair which create a long-horizon credit assignment problem

Still largely open, no end-to-end DRL method offers hard feasibility guarantees for globally-constrained COPs.

Open challenges

Robustness & Safety

- DRL policies offer no bounds on solution quality or worst-case performance.
- Black-box behavior is unpredictable outside the training distribution
- In healthcare, transportation, and energy systems, a suboptimal or unsafe decision can be costly — or dangerous
- Robustness to distributional shift and adversarial perturbations remains poorly understood

Safe-RL techniques exist in the broader RL literature, but their integration with neural combinatorial optimization is still in its infancy.

Open challenges

Interpretability

- Deep RL policies are black boxes by nature — and attention/GNN architectures only deepen the opacity
- Practitioners need to trust and justify a routing or scheduling decision. It is not sufficient for them to see that it scored well
- There some emerging framework on explainable optimisation like (Goerigk & Hartisch, 2023; De Bock et al., 2024)

LLM-based optimization frameworks are showing promise beyond modeling explaining decisions, incorporating human feedback, and producing interpretable reasoning traces (Jiang et al., 2025 / 2026a / 2026b; Shi et al., 2025).

Code walkthrough - Job Shop Scheduling Problem (JSP)

Code walkthrough - Capacitated Vehicle Routing Problem (CVRP)

Deep Reinforcement Learning for Combinatorial Optimization

<https://research.tue.nl/en/publications/deep-reinforcement-learning-for-combinatorial-optimization/>

Code repository

<https://github.com/ai-for-decision-making-tue/drl-co-tutorial/tree/main>